

QuietKey QK2

A working-draft architecture study for the air-gapped Bitcoin co-signer — synthesised from the Ian Coleman BIP39 tool, SeedSigner, ColdCard firmware, the documented loss history of this device class, and the QK2 hardware blueprints. Candidate boundaries and threat findings, not an approved subsystem.

PROJECT STATUS — PRE-GATE-B WORKING MATERIAL

Air-gapped calculator-disguised 2-of-3 co-signer — a provisioning device *and* a transient signer in one artifact. **Seed A absent at rest**, restored transiently from the cloaked encrypted Recovery Document (A1) using key A2; **provisioning** generates and verifies A, B, C, and A2, then wipes every secret from the terminal. **Seeds B and C live on bearer smart cards** (Key Card and Emergency Card); the retro-calculator terminal holds no persistent wallet secret and is interchangeable. **Camouflage is a system requirement** — artifact, document format, UI, labels, packaging — a first layer of defence, never a cryptographic factor. **QR and microSD are both required transports** (microSD is the lower-risk first path); **no USB/radio; no mechanical redesign authorized**. QR framing/capture on the existing enclosure/optics/320×240 display remains a hardware/system validation **BLOCKER**. Losing **two** signing factors can be unrecoverable. Contents are **SELECTED** product direction (not implemented), deferred, rejected, or open — nothing here is validated.

DEVICE	WALLET TYPE	DATA PATH	SOURCES READ	DATE	STATUS
QK2 co-signer	P2WSH 2-of-3 (selected)	QR + microSD (both)	3 repos + 2 PDFs	2026-08-17	Pre-Gate-B

2. [Architecture blueprint](#)
3. [Best-practices extraction](#)

5. [Implementation pitfalls](#)
6. [Threat & mitigation matrix](#)
7. [Open decisions \(must pin before ship\)](#)

00 READ THIS FIRST

One machine, two roles: provisioning device and transient 2-of-3 signer

QK2 is not two different machines. It is a single air-gapped artifact that at provisioning generates and verifies the four elements, and in daily use signs as a transient co-signer. It moves PSBTs over QR *and* microSD, and it restores Seed A from the cloaked encrypted webpage A1 using the A2 key carried on the bearer cards. Settling this framing is what the rest of the architecture hangs off.

PROVISIONING ROLE

Generate and verify **four elements**: Seed A, Seed B, Seed C, and the A2 key. Encrypt Seed-A entropy into the A1 Recovery Document, write the same A2 onto both cards B and C, verify both, then **wipe A2 and all seeds** from the terminal. No persistent wallet secret remains.

TRANSIENT-SIGNER ROLE

Restore Seed A from the cloaked encrypted webpage **A1** (decrypted with **A2**, released by a bearer card), co-sign a native **P2WSH 2-of-3** `wsh(sortedmulti(2,A,B,C))` . **B and C are bearer card signers** — physical custody is authorization. PSBTs cross the air gap by **QR and microSD**.

The four elements and the three custody objects

- **A1 — Recovery Document.** An ordinary printed webpage (a recipe, a guitar tab, travel advice) whose *footer resembles the page's own source URL* and carries authenticated ciphertext of 256-bit Seed-A entropy. Encryption is **mandatory**. No naked BIP-39 words, no conspicuous `cv0 /token`, no crypto label; safe left visibly among ordinary papers.
- **A2 — A1's decryption key.** A fresh 256-bit random key generated at provisioning, **duplicated onto both cards B and C** and

wiped from the terminal. It is not a Bitcoin signing key; it is exportable from a bearer card by design.

- **B — Key Card.** A bus-pass-like smart card holding signer B as a least-authority **account xprv at m/48'/0'/0'/2'** (+ chain code + origin fingerprint) + A2 + descriptor D. The card derives only non-hardened receive/change children and signs; it does **not** implement BIP-39/PBKDF2, and no root mnemonic is ever loaded onto or exported from it. The normal daily signer.
- **C — Emergency Card / Kit.** A second bus-pass card holding an *independent* signer C — again as an account xprv at m/48'/0'/0'/2' , not a mnemonic — plus the **same** A2 and the same descriptor D, packaged per kit tier with inheritance materials.
- **Device — retro-calculator terminal.** Holds no persistent wallet secret and is **interchangeable**; at provisioning it generates A2 with A1, writes the same A2 to B and C, verifies both, and wipes A2 and all seeds. Terminal possession is not a spending factor and there is no card-to-terminal pairing.

STOP-SHIP RISK · FUND LOSS

Losing two signing factors can be unrecoverable — state it plainly. The 2-of-3 `wsh(sortedmulti(2,A,B,C))` tolerates the loss of any one factor: B lost → recover with A1+C; A1 lost → B+C (A bypassed); C lost → A1+B; terminal lost → just replace it with another verified terminal or the open rescue image. But 2-of-3 here is **storage and recovery resilience, not two-device authorization**, and losing *two* of the three signing factors leaves funds permanently unspendable. A lost or compromised factor is **never "reissued"**: any signing-factor replacement changes the descriptor and therefore the funded wallet, so the single rule after any loss is — spend once with a surviving pair, provision a completely fresh A' / B' / C' / A2' / D' kit, and sweep the whole balance. Descriptor D lives on **both B and C** plus one redundant A2-encrypted cloaked kit copy; because every valid spending pair contains at least one card, every valid pair can recover the wallet map. Timelock, cloned-B, encrypted-B-export, and single-sig were considered and **REJECTED** (short appendix only).

Everything below honours this settled model — the disguised terminal, A1 as an ordinary encrypted webpage, the bus-pass bearer cards, and neutral kits — and maps the three reference codebases onto it.

A layered co-signer with a UI-free, headless-testable crypto core

The one structural principle all three references converge on: the security-critical core (entropy, PSBT parse/validate, sign) must be a side-effect-bounded, UI-free library (not fully deterministic — entropy acquisition and signing are not) that can be exercised headlessly against test vectors. Rendering, navigation, and I/O sit above it and never reach into key material.

UI & state machine · calculator decoy

L8

Explicit finite-state graph (SeedSigner Destination/back-stack model). Card-absent boots a fully working calculator; card insertion is the sole gate into wallet mode. Fraud detections are dead-end screens that discard the PSBT and clear history. Full addresses render on two lines, never truncated; the approve control appears only after the user scrolls the entire address and every output.

Card session protocol — minimal bearer applet

L7

Two card roles (B and C), the same minimal applet: GET_INFO / GET_POLICY / GET_A2 / SIGN / PROVISION_BEGIN·DATA·COMMIT·ABORT (and an optional Gate-B ATTEST). **No PIN / OAC / duress / retry-counter / lockout / pairing** — B and C are bearer objects, physical custody is authorization, and the 2-of-3 policy is a storage/recovery control, not a second authorization device. The card holds only the account xprv (+ chain code + origin) — it does not implement BIP-39/PBKDF2 and no root/mnemonic ever leaves it; A2 is exportable by design. **No custom card secure channel in v1** — no shared fleet key, no per-terminal pairing, no bespoke ECDH/AES/HMAC channel; the direct card bus is part of the trusted terminal boundary in v1, and a standard card-authenticated public profile (e.g. an SCP11 variant without host-shared secrets) is possible LATER hardening only, never a recovery dependency. Attestation is a manufacturing-security question, not user authentication. Monotonic counters are telemetry, not clone-detection; anti-exfil is DEFERRED (not a v1 blocker). Retirement is an on-chain sweep + physical destruction — "secure wipe" is not a reliable flash-erasure guarantee. A documented open CCID/ISO-7816 rescue CLI + reproducible rescue image ship alongside, so recovery never depends on QuietKey hardware alone.

microSD / FAT I/O (ColdCard model)

Binary PSBT v0 only (no base64/hex autodetection, no output-encoding symmetry). Read-only mount, copy-into-RAM before parse,

L6

hardened/pinned FatFs with backported 2026 CVE fixes, corrupt-card $\equiv$ absent-card, never overwrite input, `-signed.psbt / -part.psbt / .txn` naming, sync + read-back verify. No "secure delete" is offered — flash translation and wear-levelling make overwrite-and-delete an unreliable erasure claim; the input is simply left untouched.

L5

PSBT engine (ColdCard-grade validation)

Streaming offset-proxy parser, strict ordered validation, CVE-2020-14199 triple defence, change proven by full-descriptor ownership, checked-arithmetic fee guard (absolute fee + sat/vB, versioned warnings, conservative emergency ceiling, no universal 10%-of-spend rule; unknown $\Rightarrow$ fail-closed), sighash default-deny (SIGHASH_ALL only in v1), anti-TOCTOU binding of the approved canonical review-object identity re-checked before signing (raw-byte integrity checked separately).

L4

Recovery-capsule & OCR ingestion

Bounded OCR $\rightarrow$ custom base32 (QKRD-1) $\rightarrow$ AEAD-capsule pipeline. The **crypto capsule is FROZEN**: ChaCha20-Poly1305 (RFC 8439), document key HKDF-SHA256(IKM=A2, salt=wallet_id, info="QuietKey/A1/v1"), stored 96-bit nonce, 32-byte Seed-A plaintext, 128-bit tag, versioned AAD binding wallet_id. Only the *physical* layer — RS parameters, font, footer length/grouping, OCR thresholds — stays a Gate-A UNFROZEN decision. The replaceable terminal performs the AEAD decryption after a bearer card releases A2; RS correction is never an authenticity gate. Uncertainty becomes erasures (never guesses), fail-closed on any auth-tag mismatch, wallet fingerprint shown post-decrypt, closed-loop scan-back at provisioning. See the Recovery Document flow.

L3

Entropy engine (provisioning)

The **terminal** conditions all four values — Seed A, Seed B, Seed C, and the A2 key — each with its own QKEC-1 purpose (Seed-A / Signer-B / Signer-C / A2) and fresh acquisition (B and C are NOT derived from A or one master; a card contributes an RNG sample in Multi-source mode but never secretly generates B/C). Fixed profile is **24 words / 256 bits** for A/B/C; A2 is an independent 256-bit key. Two production modes only: Multi-source HKDF-SHA256 and Advanced = exactly 100 private d6 rolls per secret (400 total); the EXPERIMENTAL CloakGrid 64-tile permutation carries no production entropy claim. Terminal-side checks stay *gross* (API timeout, all-zero/repeated blocks, impossible status) — real RCT/APT belong in the raw noise source, not on an already-conditioned 32-byte result — plus build-time symbol/reachability checks that prove the intended source actually reaches the conditioner.

L2

Crypto core (UI-free Rust library + pinned libsecp FFI)

Main implementation hypothesis is **Rust** for the core, parsers, state machines, recovery codec, and transport decoding, with a narrow pinned `libsecp256k1` C FFI for secp256k1 only (an all-C core is a *measured* fallback if the ARMv6/Linux toolchain fails Gate C; Python/JS are forbidden at the secret boundary). BIP32/39, **exact deterministic RFC6979** ECDSA on the terminal, strict DER, low-S canonical form (sourced independently), verify-before-release; low-R is an optional size-only optimization; anti-exfil is a DEFERRED future hardening; no dynamic allocation in the signing path; fixed static key buffers with `memset-on-drop`.

...state handling, no dynamic allocation in the signing path, fixed state key, secure state memory on chip.

Boot & root of trust

L1

Read-only eMMC root, signed offline update packages with indexed/rotatable keys, monotonic anti-rollback floor, reproducible builds with published digests. See the honesty note on the CM0 trust gap.

Hardware reality

L0

Pi CM0 (512 MB, no swap/zram, RAM-headroom *unproven*), 320×240 ST7789, Camera 3 (two mode-locked uses only: A1 footer capture and PSBT QR capture — never auto-detects a seed QR), ISO-7816 smart card = the secure element, no USB/radio, no active tamper detection, primary AAA cells.

PRIVILEGE ARCHITECTURE · THREE PROCESSES, NOT ONE

The "core vs. shell" split is not enough if both live in one process. QK2 runs **three processes**: **qk-core** — the trusted wallet process that owns the keypad, framebuffer, card sessions, and all secret memory during wallet mode; parses/validates hostile PSBT bytes; builds the exact on-screen review; reads the physical approval directly; signs, verifies, finalizes, and returns non-secret bytes. **qk-io** — an *unprivileged* transport broker that reads the selected SD file or camera frames, enforces a first outer size/time/frame cap, passes immutable bytes to **qk-core** over a minimal local IPC, and writes only a core-provided result; it has no card access, no keypad access during approval, and no secret API. **qk-decoy** — the calculator application, with no wallet/card APIs and no wallet state, terminated before **qk-core** takes exclusive control of the display, keypad, and card slot. Process isolation limits accidental and low-privilege compromise; the Linux kernel, booted image, and physical display/keypad path remain trusted and are **one authorization trust domain** — it does not turn a compromised kernel into a safe platform.

CROSS-CUTTING REQUIREMENT · THE CLOAK

Camouflage is a system-level layer, not a threat-model footnote. It spans the whole stack: the terminal disguised as a retro calculator, A1 as an ordinary printed webpage whose footer resembles its own source URL, the bearer cards shaped like bus passes, and neutral inheritance kits. Stealth and hiding-in-plain-sight are a *first layer of defence* — NEVER a cryptographic factor and never a substitute for the 2-of-3 threshold. Every artifact, document format, UI label, and package

cryptographic vector and never a candidate for the — or a universal every, always, document format, always, and passing — must read as unremarkable to an ordinary observer.

1.1 Crypto core — the non-negotiables

Port `trezor/python-mnemonic` 's exact algorithm (or reuse its published vectors) so mnemonics interoperate bit-for-bit. Hard-code BIP-39 at **2048 PBKDF2-HMAC-SHA512 rounds** with NFKD on both mnemonic and passphrase — never expose ColdCard/Coleman's user-selectable rounds footgun. Drive all entropy base-conversion from **one declarative table** (Coleman's `eventBits` pattern), not six hand-written branches. Signing draws from ColdCard's discipline but pins the construction: **exact deterministic RFC6979** ECDSA on the terminal (strict DER, low-S, verify-before-release), optional low-R grinding as a size-only optimization, zeroise each private key immediately after use, and keep device-added signatures in a separate structure from host-supplied ones so provenance can never be confused.

SIGNING-TIME DEFENCE STACK

The v1 terminal construction is **exact deterministic RFC6979** — not an unspecified "hedged RFC6979," which is a different construction that would need an exact reviewed algorithm plus card support. Determinism is made safe by the surrounding checks, not by mixing in entropy: (1) **verify every signature** against the pubkey before releasing it; (2) **redundant sighash** computation compared before signing; (3) **low-S canonical form**, sourced independently (BIP-146 is withdrawn), plus robust nonce tests; (4) reproducible firmware so the RFC6979 path is auditable. Card signature behaviour must meet the same observable output rules (strict DER, low-S, verified by the terminal before release) and is a **Gate-B capability to specify and test**. Anti-exfil sign-to-contract on the bearer card's signing leg is a **DEFERRED optional future hardening, not a v1 requirement**: it requires the card to expose caller-controlled nonce operations that generic one-shot JavaCard ECDSA does not, and the reference `secp256k1-zkp_ecdsa_s2c` module is itself *experimental with an unstable API*. Treat it as deferred until a real card capability matrix proves it feasible and an audited implementation exists; a rigorous selective-abort/retire policy (not ">100 aborts") is part of that work.

1.2 Entropy engine — provisioning modes, never blurred

24 words / 256 bits is the only production profile for Seed A, B, and C — no 12-word / 128-bit mode exists in production (it survives only as a labelled historical regression fixture); the A2 key is an independent 256-bit random value. All four are domain-separated with fresh acquisition — B and C are never derived from A or one master. ColdCard's central lesson is that verifiable dice and hybrid mixing *cannot be the same flow*: a hybrid advertised as "verifiable dice" is security theatre; a silent TRNG-only fallback shifts all trust onto unaudited silicon. Two modes ship below; a third — the EXPERIMENTAL CloakGrid 64-tile permutation (64! » 256 bits) — carries **no security claim** until acquisition, extractor, bias, OCR, loss-detection, and human-trials are validated. The 2026 ColdCard lesson: prove the intended entropy source actually *reaches* the conditioner (build-time symbol/reachability checks, fault injection, a reproducible pure-physical mode) — health tests alone can't prove it.

Mode A — Verifiable pure dice (Advanced)

seed = SHA256(exact 100 ASCII '1'-'6' digits, no separators) , matching the de-facto ColdCard/SeedSigner hashing convention (*not* Coleman's 6→0-remapped hash — that silently produces a different mnemonic). **Exactly 100 private d6 rolls per secret** (400 total across A/B/C/A2); 99 rolls ≈ 255.9 bits is explicitly *not* the 256-bit threshold, and the Coleman/SeedSigner/ColdCard-family 99-roll figure is rejected. Invalid faces are refused immediately (never filtered), transcript reuse is a hard error, per-face bias is flagged, a live roll count / grouped transcript review / short commitment is shown, and a stark warning marks the on-screen digest as wallet-equivalent secret material. Ship a stdlib-only offline verifier like ColdCard's `rolls.py` that reproduces the same 24 words / A2 digest.

Recommended — Multi-source (QKEC-1)

seed ← QKEC-1 : a length-framed **HKDF-SHA256 conditioner** —
salt = SHA256("QuietKey/QKEC-1" || purpose || ceremony-id) ,
info = "QuietKey/256-bit-output" , run *independently* for each purpose Seed-A / Signer-B / Signer-C / A2 — over fresh samples from the SoC HWRNG plus each card's SE-RNG plus *optional* user dice, **not** a bare concatenation-hash and never one master expanded into all four. A sound conditioner means the output is computationally unpredictable *if at least one contributing source is independent, uncompromised, and non-adaptively acquired with sufficient entropy* — this is **not** the same as "entropy adds", which needs independence/correlation assumptions concatenation alone does not prove. Count only the dice bits on the user-facing meter. Using ≥2 sources helps only insofar as they are genuinely independent — being two API calls does not establish independence, which must be justified per the QK2 threat model.

Keep the *terminal* checks gross: API timeout, all-zero or identical-repeated blocks, impossible status codes, and a cheap stuck-at check (ColdCard's `len(set(seed))>4`). Do **not** claim that Repetition-Count / Adaptive-Proportion tests on an already-conditioned 32-byte API result prove SP 800-90B compliance — real RCT/APT with a documented symbol model and min-entropy assessment

belong inside the raw noise-source implementation, and the release must additionally prove call reachability in the binary (link maps + disassembly archived, fault injection disabling each source in turn). A failed source is excluded **loudly** and generation blocks — never a silent continue. Reject invalid keypresses at entry; never silently filter after the fact (Coleman filters and shows a banner — dangerous on a device where a missing roll goes unnoticed). Dice mode gates on the **exact 100-roll transcript** per secret, not a variable bit counter; Multi-source blocks until every purpose has fresh samples from its designated sources.

1.3 Boot & the CM0 trust gap — stated honestly

KNOWN LIMITATION

The Raspberry Pi CM0 has **no immutable secure-boot ROM** usable the way ColdCard's bootloader-enforced ECDSA + SE-held whole-flash checksum works. Anyone reaching the internal `nRPIB00T` pads can reflash arbitrary firmware, and there is no "genuine" indicator driven by hardware the application CPU can't forge. Do not promise ColdCard-grade verified boot. Mitigate what is possible — signed update packages, indexed keys, anti-rollback floor, reproducible builds, tamper-evident packaging — and document plainly that evil-maid reflash is out of scope so owners keep physical custody. The disguised-calculator camouflage protects against *discovery*, not against a swapped look-alike device.

1.4 Quantum-prepared, not quantum-proof — a cross-cutting posture

The vault is engineered for long dormancy under a future cryptographically-relevant quantum computer without over-claiming.

Prepared: 256-bit seeds and encryption, dormant P2WSH keys and descriptors kept offline, strict no-address-reuse, and a committed path to migrate to future PQ Bitcoin standards.

QUANTUM BOUNDARIES, STATED HONESTLY

24 words protect against seed brute-force, **not** against Shor breaking ECDSA; 2-of-3 is redundancy, not a quantum class. Native **P2WSH** gives good long-exposure protection because the output commits only to the SHA-256 of the witness script — but it *reveals* the script and keys when spent (a short mempool window). Do **NOT** use Taproot for the vault (P2TR exposes

an EC output key immediately). **BIP-360 / P2MR is DRAFT** and gives no short-exposure protection without PQ signatures — MONITOR / deferred. NIST ML-DSA/SLH-DSA exist but are not Bitcoin consensus; do not bolt an off-chain PQ signature onto a PSBT and market it as protection, and do not freeze a proprietary seed-to-PQ derivation now — reserve versioning and benchmark larger PQ payloads. **Migration trigger:** sweep from the hidden-key P2WSH outputs when a credible, reviewed PQ output/signature path deploys or CRQC progress shortens the horizon. In Quantum Shelter mode the signed output is a **finalized raw transaction** — no PSBT xpub/origin metadata goes back online.

02 BEST-PRACTICES EXTRACTION

What to take from each reference, with citations

Concrete, code-level takeaways checked against the actual repositories — not general principles.

Ian Coleman BIP39 — entropy accounting done right

Unbiased variable-length dice extraction

`entropy.js · eventBits`

The issue-#435 fix maps each d6 (after 6→0 remap) through

`{0:00,1:01,2:10,3:11,4:0,5:1}` = 1.667 bits/roll. **Never** do positional base-6→binary conversion: 6" isn't a power of two, so high bits bias and leading zeros vanish (the pre-0.5.0 bug where a single

Count emitted bits, not $\text{rolls} \times \log_2 6$

`index.js · showEntropyFeedback`

Display Total Bits from the actual binary length and gate on `bits ≥ target`. State the rule to the user ("each roll adds 1–2 bits; ~77 reach 128") so the honest discount reads as a feature, not the bug users file (#636).

Echo the interpreted entropy back

`index.js · showChecksum / showWordIndexes`

Show the full ladder: rolls → filtered digits → bits in 11-bit groups → word indexes → words → checksum. Apply the same "echo interpreted input back for audit" idea to the OCR footer reader.

"5" claimed 3 bits).

Checksum is computed, never supplied

`jsbip39.js · toMnemonic/check`

Append checksum = first ENT/32 bits of SHA256(entropy); enforce `ENT%32==0`.
Provide both directions — generate-with-checksum and verify-with-nearest-word diagnostics.

Also copy: the layered warnings tied to concrete failure modes; the debounced, fully stateless recompute-from-canonical-buffer (never incrementally patch derived displays). **Explicitly avoid:** auto-detecting the entropy base (mode is fixed by the QK2 flow); the 3,871-line DOM/crypto monolith; the DOM-bound global PBKDF2 rounds; `zxcvbn` "time to crack" (a password estimator, misleading for raw entropy — show counted bits only); and the excess-bit truncation whose code contradicts its own comment.

SeedSigner — statelessness & hostile-stream ingestion

Statelessness by structural impossibility

`models/seed_storage.py`

Seeds live in one RAM-only container with a pending→finalize→discard lifecycle and **no serialization method at all**. Make persistence impossible to write, not merely disabled.

Home screen = hard state-flush

`controller.py:294–305`

Every arrival at the main menu nulls all flow state and the back-stack. Designate one node as the guaranteed ephemeral-state flush; route fraud dead-ends through it with `clear_history`.

Type-locked, strategy-per-format decoder

`models/decode_qr.py`

One facade, first-frame type lock-in, a decoder strategy per format, "does the library actually parse it" as the validity test, and a hard exception on mid-stream type change. The reader shape for the OCR footer stream.

Fountain reassembly with honest progress

helpers/ur2/

Lock message-len/checksum/fragment-len from the first frame; verify whole-message CRC-32 before use; per-frame status feedback. Vendored, not a dependency — keeps the most security-critical parser auditable.

Closed-loop backup verification

seed_views.py:1595–1690

After transcribing a SeedQR, scan it back and compare word-for-word. Apply the identical loop to the QK2 Recovery Document before first funding.

MVC layering & headless flow tests

docs/code_structure.md

Models (no rendering) / Views (return Destinations) / Screens (pure renderers) / Controller. One global exception boundary converts any parser crash into a clean error screen with history flushed.

Heed SeedSigner's own warnings: its `data[3:] OP_RETURN` slice assumes `OP_PUSHDAT1` is always present and shifts short payloads — parse push opcodes properly. CPython cannot zeroise; QK2's C core must do materially better. And note SeedSigner has *no* microSD-PSBT path — for that, ColdCard is the only model.

ColdCard — PSBT robustness & microSD handling

Streaming offset-proxy PSBT parser

shared/psbt.py · psbtProxy

Store `(offset,length)` tuples, not values; re-read lazily in small chunks; never fully deserialize the unsigned tx. Signs a 2 MB / 2000-output PSBT on a microcontroller.

CVE-2020-14199 triple defence

history.py · OutptValueCache

(a) persistent outpoint→amount cache re-checked across sessions; (b) recompute `calc_txid(non_witness_utxo)==prevout.hash` ; (c) accept `witness_utxo` alone only when provably segwit, else refuse to show a fee — but QK2's own PSBT v0 profile tightens this to require the full previous transaction for every input, with no witness-only shortcut. See the statelessness caveat in Pitfalls.

Change = reconstruct, never trust

shared/psbt.py · validate/sign_it

Rebuild the witness script from the *registered* wallet (BIP-67 order), hash-compare to the scriptPubKey, then re-derive the pubkey from the live seed at signing time — deliberately twice, cheap-then-authoritative — raising a distinct `FraudulentChangeOutput` class.

Anti-TOCTOU binding

shared/auth.py:390–647

Hash the staged PSBT at parse; re-check hash + write-counter immediately before the first sighash with no interactive yield between; wipe the region on mismatch.

SD lifecycle & never-overwrite naming

shared/files.py · CardSlot

Context-managed mount/unmount with power-down on exception; copy-in before parse; `pick_filename` appends `-N` and never overwrites; corrupt FS $\equiv$ `CardMissingError`. QK2 **does not adopt** ColdCard's secure-blank offer — flash wear-levelling makes overwrite-and-delete an unreliable erasure claim, so the input is simply left untouched.

Checked fees & default-deny sighash

shared/psbt.py · `consider_*`

ColdCard shows negative fee fatal and a configurable $\geq 10\%$ / $\geq 5\%$ percentage tier — QK2 **rejects the universal 10%-of-spend rule** (consolidations and small sends make it unreliable). Instead: checked 64-bit arithmetic, explicit *absolute fee + sat/vB* always displayed, versioned warning policy, and a conservative absolute emergency ceiling; uncomputable/negative fee is fatal and unknown-fee is named, never shown as zero. NONE/SINGLE blocked by default; SIGHASH_ALL only in v1; unknown flags fatal.

Security-model elements portable to QK2: split-trust across independent secure elements (QK2 realises this as the 2-of-3 A/B/C threshold, not PIN/duress features — those are rejected under the bearer-card model); BIP-85 as a derived-secrets primitive (child seeds, test vectors); firmware signature enforcement in an *immutable* layer with numbered keys and a version high-water mark; and the repo layout itself — `shared/` portable core, `stm32/` bootloader, `unix/` simulator, `testing/` pytest, `docs/` threat-model prose + stdlib verifier scripts. **Licensing caveat:** ColdCard firmware is MIT *plus Commons Clause*; QK2 must use a documented clean-room implementation of the learned patterns and must **not** copy ColdCard code into a commercially sold product that it then describes as unrestricted open source.

the design, engineered against foreseeable mistakes — measured, not "foolproof"

"Fool-proof" is not an attainable property and is not claimed here. QK2 targets *measurable* acceptance criteria — mistake-resistant, bounded, fail-closed, scroll-gated review, closed-loop scan-back, zero unauthenticated plaintext release under fuzzing — and states its residual risk plainly: a booted terminal that is malicious or has malicious firmware is one authorization trust domain and can obtain the normal threshold during an ordinary A1+B session. It cannot guarantee the absence of implementation defects.

THREE VALID SIGNING / RECOVERY FLOWS

Normal — A1 + B: decrypt Seed A from the A1 document with A2 (released by card B), then sign with A + B. **B lost — A1 + C:** decrypt A with A2 from card C, then sign with A + C. **A1 lost — B + C:** sign with B + C directly (A bypassed). **Terminal lost** → replace it with another verified terminal or the open rescue image; no on-chain rotation unless compromise is suspected. Any lost or compromised *signing factor*, however, means: spend once with the surviving pair, then provision a fresh A' / B' / C' / A2' / D' kit and sweep — a factor is never reissued, because replacing any key changes the descriptor. Losing *two* signing factors can be unrecoverable — state it plainly.

Flow A — Provision the wallet & generate the Recovery Document

01 Insert bearer card → leave calculator mode

Card-absent, the device is only a calculator (keypad LEDs hardware-dark). Insertion of a bearer card (B or C) is the sole transition; the card's monotonic counters are shown so a swap/clone is evident. No PIN, no duress prompt — physical custody of the card is the authorization, and the 2-of-3 policy is the spending control.

GUARD · wallet UI gated only by a present bearer card; no PIN/lockout to forget

02 Choose entropy mode

Verifiable pure-dice, or domain-separated hybrid. The choice is explicit and its trade-off stated on screen — verifiability vs. observer-resistance.

GUARD · no silent TRNG-only fallback; mode is never inferred

03 Collect & health-test entropy

Every keypress echoes immediately; invalid keys are refused, not filtered. A live bit counter gates continuation; per-face bias and SP 800-90B tests run continuously. A failed source is excluded loudly and blocks generation.

GUARD · ≥ 2 independent sources; dice can only strengthen the seed

04 Provision A, B, C, A2 & display the wallet fingerprint

The terminal conditions **four domain-separated secrets** — Seed A, Seed B, Seed C, and the 256-bit A2 key — each from fresh acquisition (B and C are never derived from A). From logical Seed B/C it derives the **BIP-48 account xprv at m/48'/0'/0'/2'** (+ chain code + origin fingerprint) and provisions *only that least-authority material* to each card, then wipes the root/mnemonic — the applet implements no BIP-39/PBKDF2. The **same A2 is duplicated onto both B and C**, then wiped from the terminal. The descriptor for the **SELECTED** policy — native P2WSH `wsh(sortedmulti(2,A,B,C))` — is assembled from all three cosigner xpubs; the device shows a compact policy identity / descriptor checksum (e.g. `#kuyqfyj3`) plus the first receive/change addresses to record. `wallet_id = SHA256(recv_desc || 0x00 || change_desc)`. The **complete recovery metadata (descriptor D)** — every cosigner xpub, all key origins, threshold, script type, network, derivation, checksum, birthday — is written to **both B and C** plus one redundant A2-encrypted cloaked kit copy, so every valid pair (each contains a card) can rebuild the wallet map.

GUARD · derive-and-compare every A/B/C origin/xpub from the real factors; a checksum is error-detection, not authentication

05 Emit & verify the cloaked Recovery Document (A1)

A1 is printed as an ordinary webpage whose footer resembles its own source URL and carries **authenticated ciphertext** of the 256-bit Seed-A entropy — **encryption is mandatory**. The cryptographic capsule is now **frozen: ChaCha20-Poly1305 (RFC 8439, IETF)** from a reviewed constant-time implementation, document key = `HKDF-SHA256(IKM=A2, salt=wallet_id, info="QuietKey/A1/v1")`, a fresh **stored 96-bit nonce**, plaintext = exactly 32 bytes of Seed-A, AAD = magic + coding/crypto versions + network + full `wallet_id`, **128-bit tag**, and no plaintext released before the tag verifies. Only the *visual* layer — Reed-Solomon parameters, font, footer length/grouping, and OCR thresholds — stays behind Gate A. **Mandatory closed loop**: the user re-captures the printed footer; the device decrypts Seed A with A2, derives Seed A's account xpub, and confirms it matches the independently-stored descriptor D before any funds may be received.

GUARD · the AEAD tag authenticates the ciphertext; the descriptor comparison authenticates the wallet

06 Create coordinator material & verify addresses on-device

Derive and display receive/change addresses from the selected P2WSH 2-of-3 descriptor. Coordinator export is **tiered: Simple Recovery and Inheritance MAY export the full watch-only descriptor** to one explicitly supported, tested coordinator (with the privacy / future-pubkey-exposure tradeoff explained); **only Quantum Shelter is address-packs-only**, with no account xpub or full descriptor ever online. A universal "addresses-only, never descriptor" rule is too costly to be the simple path. Whatever the tier, the companion wallet is untrusted for addresses; deposits are only ever confirmed against the QK2 screen.

GUARD · defeats the companion-substitutes-address attack; xpub/descriptor disclosure is a tier choice, not a leak

Flow B — Import, review, sign, export a PSBT

01 Import over microSD or PSBT-QR (both required transports)

microSD is the lower-risk first path: mount read-only; copy the file into RAM before parsing; **binary PSBT v0 only — no base64/hex autodetection and no output-encoding symmetry**; hard cap enforced *during* streaming; corrupt card $\equiv$ absent. **QR** uses the camera's mode-locked PSBT capture only (never a seed QR) with the ONE selected framing — **BBQr file type P , uncompressed base32** (chosen because both reviewed SeedSigner and ColdCard support it, its sequential-part model is simpler to bound than a fountain decoder, and uncompressed avoids a decompression-bomb surface). The FORMAT is settled; only its *performance* on the existing enclosure/optics/320×240 display remains a HW/system validation BLOCKER (no mechanical redesign authorized).

GUARD · never parse in place on removable media; camera never auto-detects an arbitrary QR class

02 Parse & bind (anti-TOCTOU)

Streaming offset-proxy parse behind one fail-closed exception boundary; hard caps on size, input/output/signer counts, path depth. Bind the approval to the **immutable canonical review-OBJECT identity** re-checked before signing; raw-byte integrity of the staged bytes is verified as a **SEPARATE** check — the binding is *not* a parsed-region hash plus write-counter.

GUARD · every byte treated as hostile fuzzed input

03 Validate in strict order

Structure → PSBT v0 profile (v2 deferred; do not present v0/v2 as an operative either/or) → per-input txid recomputation → CVE-2020-

14199 triple defence → reject only conflicting/malformed/duplicate/already-QK2-signed inputs (a multisig PSBT normally carries another cosigner's signature, so already-signed inputs are not rejected wholesale) → reject any input the device cannot fully attribute and render (foreign inputs are out of v1) → mixed-policy rejection → checked-arithmetic fee (absolute fee + sat/vB always shown; versioned warnings; conservative absolute emergency ceiling; no universal 10%-of-spend rule; unknown/negative/uncomputable ⇒ fatal) → sighash default-deny (SIGHASH_ALL only in v1) → OP_RETURN size with correct push-opcode parsing. Fraud proofs are fatal with distinct titles; policy issues fatal-by-default; ambient facts are counted warnings.

GUARD · signing cannot begin until every check passes

04 Prove change ownership from the selected 2-of-3 descriptor

Reconstruct the full P2WSH witness script from the **SELECTED policy's complete key set** (all three cosigner xpubs — A/B/C, sorted), require `descriptor.owns(output)`, and flag owned-but-anomalous paths ("returns to your seed at an unusual path"). No skip button. Missing/zero fingerprints handled by derive-and-compare, not a dead end.

GUARD · path-pattern match on our own key alone is NOT sufficient

05 Human review — full, scroll-gated

Overview (spend/change/fee + I/O pictogram) → explicit arithmetic (in – recipients – fee = change) → each recipient's **full** address in 4-char groups over two lines → each change output with path + verification badge → raw OP_RETURN. The approve control materialises only after the user scrolls the *entire* address of every output.

GUARD · defeats address-poisoning & middle-group vanity collisions

06 Re-verify, then sign

Re-check the immutable canonical review-object identity (and, separately, raw-byte integrity) with no interactive yield; the transient Seed-A leg signs with **exact deterministic RFC6979** ECDSA + low-S canonical form and robust nonce tests (anti-exfil is a DEFERRED optional future hardening, not a v1 requirement); the bearer card (B or C) signs its leg meeting the same observable output rules (its exact nonce behaviour is a Gate-B capability to specify and test); the device **verifies both signatures** against their pubkeys before emitting anything. Two of the three factors satisfy the 2-of-3 threshold (A+B, A+C, or B+C).

GUARD · approve-X-sign-Y is closed; a faulted/garbage sig never leaves

07 Export deterministically

Emit the signed result as a NEW object (input untouched); verify existing partial signatures and **add only the required fields** — preserve

valid unknown/cosigner PSBT key-values, never gratuitously rewrite; canonical field ordering; temp → sync → validate → rename → cleanup, then read-back verify; retry on transient write error so a completed signature is never lost. **Every completed local 2-of-3 session returns a finalized raw transaction;** a finalized PSBT (<name>-signed.psbt) is also returned where the selected tier/coordinator needs it. The input file is left untouched — no "secure delete" is offered, since flash wear-levelling makes it an unreliable erasure claim. The same result may leave over the QR path on the selected BBQR profile; in Quantum Shelter mode *only* the finalized raw transaction goes to the online side, with no PSBT xpub/origin metadata returning online.

GUARD · a deterministic output closes signature/encoding covert channels

AT ANY POINT

Card removal or inactivity timeout aborts and wipes. Battery pull is a safe abort because all secrets are RAM-only; the SD writer stages temp → sync → validate → rename → cleanup so an interrupted write leaves the original input intact, rather than relying on a claim that no partial write ever lands. Warning screens can't be revisited via Back; fraud screens discard the PSBT and flush history.

04 IMPLEMENTATION PITFALLS

Mistakes that have cost real people real money

Each of these is a named, documented failure in this device class — not a hypothetical. Combining dice entropy, OCR/QR streams, and FAT parsing multiplies the surface, so treat every one as mandatory.

P1 A PRNG (or a single trusted TRNG) standing in for real entropy

Every headline loss traces here: Milk Sad / libbitcoin bx (CVE-2023-39910, mt19937 seeded by a 32-bit clock), Trust Wallet (CVE-2023-

31290, ~\$7M), Profanity (Wintermute ~\$160M), the 2013 Android SecureRandom nonce reuse, and ColdCard's own predictable-RNG fallback + 32-bit reseed regression. A single narrow function silently substituting a PRNG compromised every downstream key.

Mitigate: physical dice as the entropy floor; hybridise through the **QKEC-1** framed HKDF conditioner over the two DESIGNATED hardware sources plus optional dice (never a bare $H(\text{dice} \parallel \text{trng})$) so a bad source can't lower entropy below the dice floor. *Assert* the hardware entropy source is active at boot (never an `#ifndef`-style existence check) and fail closed. No non-CSPRNG anywhere in the key path.

P2 Trusting host-asserted segwit input amounts (CVE-2020-14199)

BIP-143 signs the amount the host claims without binding it to the real UTXO. A lying coordinator runs two passes with different amounts and splices the signatures into one transaction that burns the difference to fees.

Mitigate: require the full previous transaction for *every* input feeding the fee check and verify `calc_txid==prevout.hash`; the QK2 PSBT v0 profile permits **no witness_utxo -only shortcut** — a `witness_utxo` alone is never sufficient, even when provably segwit.

Statelessness caveat: ColdCard's durable defence — a persistent outpoint→amount cache — is impossible on a stateless device, so QK2 mandates full-prevtx for all inputs and may additionally seal a signed spend-set commitment onto the card so a second pass is detected.

P3 Multisig change fraud — "my key is in there" is not enough

In any multisig, an attacker presents change whose script embeds the device's real key *plus* a cosigner xpub the attacker controls. A device that only confirms "my key at a change path" labels it safe change and siphons funds on every spend (the ColdCard $\leq 3.1.9$ / benma xpub-swap class).

Mitigate: reconstruct the full witness script from the selected 2-of-3 descriptor (its **complete key set** — A/B/C, sorted), require `descriptor.owns(output)`, and make verification mandatory — no skip path. A descriptor checksum is error-detection, *not* crypto-authentication (an attacker can build a different valid descriptor with a valid checksum), so derive and compare every A/B/C origin/xpub from the real factors. Verify the child xpub (hardened derivation), not just the master fingerprint.

P4 Nonce kleptography — covert seed exfiltration through valid signatures

Backdoored terminal firmware (the transient Seed-A leg) or a malicious bearer card (its B/C signing leg) biases ECDSA nonces to encode key bits into R across a handful of signatures. The signatures are valid, leave over microSD, land on the public chain, and the attacker who knows the backdoor recovers the key — no network ever used.

Mitigate (v1): exact deterministic RFC6979 ECDSA (not an unspecified "hedged RFC6979," which is a different construction needing its own reviewed algorithm), low-S canonical form (sourced independently), robust nonce tests, verify every signature before release, and ship reproducible firmware so the RFC6979 path is auditable. The card leg meets the same observable output rules with its exact nonce behaviour specified/tested at Gate B. Anti-exfil sign-to-contract on either leg is a **DEFERRED optional future hardening**, not a v1 requirement — generic one-shot JavaCard ECDSA likely cannot expose the caller-controlled nonce it needs; QK2 tracks it as an open item, not a ship blocker.

P5 Treating reassembled QR/OCR data or a mounted FAT image as trusted

The 2026 runZero FatFs disclosures make mounting a crafted card an unattended RCE — CVE-2026-6682 (FAT32-mount integer overflow → code exec), -6687 (`f_getlabel` overflow), -6688 (LFN overflow), most unpatched upstream. Unbounded fountain / OCR streams can exhaust the unproven 512 MB. ColdCard's own 5.5.1 fixes for malformed NDEF/JSON/7z show what crashes look like.

Mitigate: pin a hardened FatFs with backported fixes; mount read-only, `noexec/nosuid/nODEV` ; length-validate labels and filenames in the wrapper before any copy; corrupt ≡ absent (no on-device recovery, offer destructive format only); cap total size / part count / fragment size *before* buffering; one fail-closed exception boundary that zeroes; fuzz the PSBT/UR/OCR chain off-target.

P6 Truncated address display & the OP_RETURN slice bug

Address poisoning grinds look-alikes matching prefix+suffix (CMU counted 270M attempts in two years) to beat the truncation habit. Separately, SeedSigner's `data[3:]` OP_RETURN slice assumes OP_PUSHDATA1 and shifts short payloads, so the user approves bytes different from what's displayed.

Mitigate: the device screen is the only trusted surface — render full, chunked, scroll-gated addresses for every output, never a truncated form; support on-device comparison against a verified address. Parse OP_RETURN push opcodes correctly (direct push vs. OP_PUSHDATA1/2), never a fixed slice.

P7 Firmware downgrade over the signed-update path

An authentically-signed but *older* image containing a since-patched bug passes signature verification and rolls the device back to an exploitable version. Signature validity is necessary but not sufficient.

Mitigate: a monotonic anti-rollback high-water floor enforced as low as the CM0 allows (not in updatable app code), indexed/revocable signing keys, and user-present confirmation showing the exact version being installed. Given the CM0 trust gap (§1.3), document the residual risk honestly.

P8 Seed residue in RAM, swap, DMA, or framebuffer

Statelessness only helps at *rest*. In-use, Seed A and leg-A keys can survive in swap/zram, core dumps, camera/CSI DMA buffers, the display framebuffer, or GC-managed heap — recoverable by a later eMMC image or cold-boot attack.

Mitigate: enforce no-swap/no-zram at build; disable core dumps, ptrace, and secret paths in journald/dmesg/tmpfs; `mlock` key buffers; keep keys in fixed static buffers with compiler-barrier `memset -on-drop` and no heap copies; wipe CSI DMA buffers after scan and the framebuffer after seed words are shown; zeroise on power-off/timeout/abort *and verify the wipe*.

P9 Cross-tool dice incompatibility & weak entropy stretched to long mnemonics

Coleman's fixed-length mode hashes the 6→0-*remapped* string, so identical rolls yield a different mnemonic than ColdCard/SeedSigner (which hash raw ASCII) — users cross-verifying conclude the device is corrupt. And hashing whitens weak input: 10 rolls can silently become a 24-word phrase carrying far less than 256 bits.

Mitigate: pick one construction byte-for-byte (SHA-256 over raw '1'-'6' ASCII), display the exact hash-input string, publish a stdlib verifier, and document the Coleman incompatibility. Make minimum entropy a *hard* gate — refuse to emit N words until $\geq 32 \cdot N/3$ real bits are collected. Never let SHA-256 masquerade as entropy.

P10 A doctored Recovery Document / OCR→RS→capsule exploit

A swapped or crafted QKRD-1 footer could target the AEAD parse path, abuse RS mis-correction to inject a chosen ciphertext, or decrypt to an attacker-known Seed A the user then funds. The *crypto* capsule is now frozen — **ChaCha20-Poly1305 (RFC 8439)**, stored 96-bit nonce, 32-byte ciphertext, 128-bit tag, versioned AAD binding `wallet_id` — so the capsule byte layout is determined; what remains UNFROZEN is only the *visual* coding math (RS parameters, base32 grouping, font, footer length, OCR thresholds), which still depends on that fixed capsule (a stored 12-byte nonce + 16-byte tag + 32-byte ciphertext + version fields sets the minimum footer length and RS code) and must be published with exact coding math and print/capture vectors before the physical layer is fixed at Gate A.

Mitigate: encryption is mandatory — treat any decoded material as fully untrusted, with exact length bounds before parsing, the frozen **ChaCha20-Poly1305 (RFC 8439)** AEAD, and its stored 96-bit nonce. Authenticity is layered: the **AEAD tag authenticates the ciphertext**, and deriving Seed A's account xpub and comparing it to the **independently-stored descriptor D + checksum authenticates the wallet**. Require the tag to verify and reject on any failure (never proceed on mis-corrected data). A2/Seed-A recovery stays independent of any single card (A2 lives on both B and C by design; the replaceable terminal performs AEAD decryption). Use constant-time handling, show the wallet fingerprint post-reconstruction, and fuzz the whole chain. The reader must convert OCR uncertainty into *erasures*, never guesses.

05 THREAT & MITIGATION MATRIX

Attack surface, ranked, mapped to the layer that stops it

ATTACK	CLASS	PRIMARY MITIGATION	LAYER
Losing two of three signing factors → unspendable	FUND LOSS	Native P2WSH 2-of-3 (independent A/B/C) selected; any one factor recoverable via A1+B / A1+C / B+C; state plainly that losing TWO factors is unrecoverable; timelock/cloned-B/encrypted-B-export/single-sig rejected	Policy
Complete descriptor /	FUND LOSS	Descriptor D (all A/B/C xpubs, origins, threshold, script, network, derivation, checksum,	Recovery

recovery metadata not surviving		birthday) stored on BOTH B and C plus one redundant A2-encrypted cloaked kit copy — so every valid pair (each holds a card) can rebuild it; wallet_id = SHA256(recv 0x00 change); kept offline + encrypted, no cloud sync	
Weak / predictable entropy	FATAL	Exactly-100-roll dice floor + QKEC-1 multi-source HKDF; gross terminal checks (RCT/APT belong in the raw noise source, not on a conditioned result); release-binary reachability proof; boot assert	L3
CVE-2020-14199 amount lie	FATAL	Full prevtx for every input (no witness-only shortcut); optional card-sealed spend-set commitment	L5
Multisig change / xpub swap	FATAL	Full-descriptor ownership proof, mandatory, checksum cross-check	L5/L7
Nonce kleptography (both legs)	FATAL	Exact deterministic RFC6979 + low-S + verify-before-release + reproducible builds; card nonce behaviour specified/tested at Gate B (anti-exfil = deferred optional)	L2/L7
Malicious FAT image (RCE)	FATAL	Hardened pinned FatFs, read-only mount, copy-in, corrupt≡absent	L6
Doctored Recovery Document	FATAL	Encryption mandatory: AEAD tag + stored random nonce authenticate the ciphertext; derive Seed A's account xpub and compare to the independently-stored descriptor D to authenticate the wallet; A2 recovery independent of any single card; fingerprint display; fuzz	L4
Approve-X-sign-Y (TOCTOU)	FATAL	Bind approval to the canonical review-object identity (raw-byte integrity a separate check); re-verify with no yield before first sighash	L5
Companion substitutes receive address	FATAL	On-device address explorer; verify deposits on QK2 screen only	L8
Address poisoning / vanity collision	HIGH	Full scroll-gated address display in 4-char groups; no truncation	L8

Non-default sighash (NONE/SINGLE)	HIGH	Default-deny with explicit danger warning; ALL required for consolidation	L5
Firmware downgrade	HIGH	Monotonic anti-rollback floor; indexed keys; version-shown confirm	L1
Glitch → nonce reuse	HIGH	Deterministic RFC6979; redundant sighash; verify-before-release; glitch monitors	L2
Seed residue in RAM/DMA/FB	HIGH	mlock, static buffers, memset-on-drop, wipe DMA/FB, verify wipe	L2/L9
Malicious terminal / firmware in an A1+B session obtains the threshold	RESIDUAL	Not solved by 2-of-3. A single booted terminal that is malicious sees Seed A + A2 and can request B's signature, so it can obtain/control the normal threshold — 2-of-3 is storage/recovery resilience, NOT two-device authorization; cards provide no independent display. The booted terminal + display path + firmware are ONE trust domain. Mitigate only by verified reproducible builds, buy-direct, tamper-evident packaging; documented residual risk	L0/L1/L2
Evil-maid implant (CM0 has no secure boot)	RESIDUAL	Tamper-evident packaging; buy-direct; documented out-of-scope	L0/L1
Fountain / OCR OOM (512 MB unproven)	HIGH	Bound size/parts/fragment before buffering; P04 bench sign-off	L4/L6
Observed / loaded dice at provisioning	MED	Hybrid mix so observed dice can't reproduce seed; bias check	L3
QR/format confusion (seed QR ↔ PSBT)	CONTAINED	Camera has two mode-locked uses only (A1 footer, PSBT QR); never auto-detects a seed QR or accepts arbitrary QR classes; ONE selected framing — BBQR type P, uncompressed base32 (performance on fixed mechanics still a Gate-A blocker)	L8

What must be pinned before production funding

These are unspecified in the QK2 documents today. Each is a place where "we'll decide later" becomes a fund-safety bug against a long-lived money artifact.

1. **Wallet policy — SELECTED; recovery to rehearse.** Native P2WSH `wsh(sortedmulti(2,A,B,C))`, 2-of-3 with independent A/B/C, is the selected product direction; timelock, cloned-B, encrypted-B-export, and single-sig are **REJECTED** alternatives (short appendix only). What remains to pin and rehearse: that the complete descriptor D survives independently, and that losing any one factor is recoverable (A1+B / A1+C / B+C) while losing two is not.
2. **Complete descriptor / recovery-metadata survival — stop-ship.** The **complete checksummed descriptor D** (two descriptors, receive + change; all A/B/C xpubs, origins, threshold, script type, network, derivation, checksum, birthday; `wallet_id = SHA256(recv || 0x00 || change)`) lives on **both B and C** plus one redundant A2-encrypted cloaked kit copy. Because every valid spending pair contains at least one card, every valid pair can rebuild D; if all three factors survive, fixed-policy D can even be reconstructed, but with one factor missing D is critical. D is privacy-sensitive — its xpubs expose EC public keys (BIP-360 warns) — so keep it **offline and encrypted at rest, no cloud sync**. Coordinator disclosure is tiered: Simple/Inheritance may export the full watch-only descriptor to a supported coordinator, while Quantum Shelter keeps D as a cloaked "wallet map" and puts only address packs online. Cards hold an *account xprv*, not a mnemonic — there is no separate secret BIP-39 word backup for B/C; A is backed up solely by the encrypted A1 capsule.
3. **The QKEC-1 conditioner, byte-for-byte** — pin the framed HKDF conditioner over the two designated hardware sources plus optional dice: exact function, input ordering and length framing, domain-separation tags, and the rule that user entropy only adds. Publish vectors.
4. **Dice construction** — name one (SHA-256 over raw '1'-'6' ASCII recommended), fix minimum rolls and bias check, ship a stdlib verifier, document Coleman incompatibility.
5. **Card ⇔ terminal session protocol** — the minimal bearer applet: GET_INFO / GET_POLICY / GET_A2 / SIGN / PROVTSTON BEGTN·DATA·COMMIT·ABORT (+ optional Gate-B ATTEST). The card holds only the account xprv (no BIP-39/PBKDF2).

no root/mnemonic export); A2 exportable by design. **No PIN / OAC / duress / retry-counter / lockout / pairing. No custom card secure channel in v1** — no shared fleet key, no per-terminal pairing, no bespoke ECDH/AES/HMAC channel; the card bus is inside the trusted terminal boundary in v1, and a standard card-authenticated public profile (e.g. an SCP11 variant without host-shared secrets) is possible LATER hardening only, never a recovery dependency. Attestation is manufacturing assurance only (real trust chain + rotation/revocation, no online-revocation recovery dependency); monotonic counters are telemetry, not clone-detection; anti-exfil deferred. Retirement is on-chain sweep + physical destruction — no "RETIRE-WIPE" is treated as reliable flash erasure. Ship a documented open CCID/ISO-7816 rescue CLI + reproducible rescue image + commodity-reader test suite so recovery never depends on QuietKey hardware. Pin ISO 7816-3/-4 framing (T=0/T=1, ATR validation) and APDU replay protection.

6. **Normative standards table** — pin BIP-32 (index bounds $0 \leq i < 2^{31}$), 39 (English, 2048 rounds fixed), 43/48 (the BIP-48 P2WSH multisig derivation for A/B/C), 67/sortedmulti descriptor order, 174 (PSBT v0 profile; v2/370 deferred), 380–389 + checksum (BIP-389 multipath is an optional Draft convenience, NOT the sole decades-long authority — keep two ordinary checksummed receive/change descriptors as the conservative authority), SLIP-132 (reject ypub/zpub in descriptors), 85 (derived-secrets primitive, if used), 340/341 (do NOT use Taproot for the vault — P2TR exposes an EC output key immediately), 360/P2MR (DRAFT — monitor/deferred), plus the ONE selected QR PSBT framing — **BBQr file type P , uncompressed base32** (SeedQR / UR2 / Specter / generic autodetect are NOT PSBT-framing candidates; only BBQr's performance on the fixed mechanics remains a Gate-A blocker).
7. **RAM-only lifecycle numbers & a language decision record** — inactivity timeout, card-removal-wipe. On language, the **main implementation hypothesis is Rust** for the core, hostile parsers, and state machines (PSBT, descriptor, OCR/recovery codec, transport decode), with a narrow pinned `libsecp256k1 C` FFI for curve math only — because these are hostile-input parsers where memory safety is a direct security property. An **all-C core is a measured fallback**, chosen only if the ARMv6/Linux toolchain fails Gate C, and then via a written decision record compensating with checked-integer helpers, fixed arenas/bounds, whole-program fuzzing, ASan/UBSan/Miri host tests, static analysis, compiler hardening, reproducible cross-builds, and a formal memory budget. Python/JS are forbidden at the production secret boundary. "No dynamic allocation in the secret path" is a target to be demonstrated with a memory plan, not an assumption.
8. **Scope** — single sealed wallet or multiple descriptors? (Regtest/testnet is dev-only, not a shipped-scope question; production firmware is mainnet-only) Coordinator interop matrix (Sparrow / Electrum / Specter / Nunchuk / Core) is for watch-only + PSBT

firmware is mainnet only.) Coordinator interop matrix (Sparrow / Electrum / Specter / Hardware / Core) is for wallet only + PSBT coordination only — **card signing still requires the documented QK2 APDU applet and the open rescue CLI**, so "recover in any generic wallet" must never be claimed for the non-exportable cards. Publish a conformance vector set including the attack corpus.

9. **P04 RAM headroom** — the CM0's 512 MB for worst-case capture+OCR+RS+PSBT is unproven and gates ECAD; the SoC HWRNG peripheral (one of two mandatory entropy sources) is literally unnamed in both PDFs.

QuietKey QK2 · architecture synthesis · 2026-08-17

Compiled from primary-source reads of iancoleman/bip39, SeedSigner/seedsigner (@5088588), Coldcard/firmware (~5.5.1), the QK2-04 & QK2-HW-P0.1 blueprints, and the disclosed loss history of this device class (Milk Sad, CVE-2020-14199, benma xpub-swap, anti-klepto, runZero FatFs 2026, Ledger Donjon / Kraken physical attacks). Where sources conflicted the divergence is stated rather than smoothed over. This is a working draft for engineering review, not a security audit.